Campus League

Plataforma web para la gestión de torneos deportivos universitarios

Asignatura	Tecnologías en Internet
Institución	Universidad Adventista de Bolivia (UAB)
Entrega	Segunda entrega: Producto implementado y repositorio GitHub
Fecha	11 de noviembre de 2026
Repositorio público	**<https://github.com/deaht911/campus-league>**

Integrantes

Integrante	Usuario GitHub
Samuel Eli Ramírez Mamani	deaht911
José Alcozer	jose-1
Roger Huarachi Rojas	roger69
Kevin Flores	Gn_Abdias

Contenido

1. Carátula e información general
2. Resumen ejecutivo y alcance
3. Trazabilidad de requisitos
4. Arquitectura del sistema
5. Diseño de persistencia en MongoDB
6. Especificación de endpoints y API REST
7. Configuración de entorno y variables
8. Manual de ejecución y pruebas
9. Conclusiones y trabajo futuro

1. Carátula e información general

Este documento corresponde a la Segunda Entrega del Proyecto Integrador. El código fuente completo está en el repositorio público indicado arriba, organizado como monorepo (client/, server/, docs/) con ramas feature/* integradas a develop y main mediante git merge --ff-only, de modo que el historial conserva a cada autor.

Elemento	Ubicación
Código del servidor	server/
Código del cliente	client/
Diagramas, capturas e informe	docs/
Documentación interactiva de la API	http://localhost:5000/api-docs (Swagger UI, OpenAPI 3.0)
Manual de ejecución	README.md (raíz) y sección 8 de este informe

2. Resumen ejecutivo y alcance

Campus League permite organizar torneos deportivos entre facultades: los estudiantes se registran con su correo institucional, crean equipos y los inscriben; el organizador aprueba inscripciones, genera el calendario y reprograma partidos; el árbitro registra el acta digital; y el público consulta calendario, resultados, tabla de posiciones y goleadores. El organizador puede exportar los reportes en PDF y los capitanes reciben notificaciones por correo.

Tecnologías integradas

Capa	Tecnología
Frontend	React 19, Vite, Tailwind CSS 4, React Router, Axios
Backend	Node.js, Express 5, express-validator, helmet, cors, express-rate-limit
Base de datos	MongoDB con Mongoose 9 (local o Atlas)
Seguridad	JWT (30 min), bcryptjs (salt >= 10), RBAC con 5 roles, OTP por correo institucional
Correo y reportes	Nodemailer (SMTP configurable) y PDFKit
Documentación	Swagger UI (OpenAPI 3.0)
Calidad	ESLint, 39 pruebas automáticas (node:test + Supertest)

Alcance alcanzado en esta entrega: los 15 requisitos funcionales (RF-001 a RF-015) y los 8 no funcionales de la primera entrega tienen implementación o justificación (secciones 3 y 8). Los 13 casos de uso están cubiertos.

3. Trazabilidad de requisitos

3.1 Requisitos funcionales (RF → código)

RF	Descripción	Componente frontend (React)	Endpoint / servicio backend
RF-001	Registro con correo institucional y OTP	RegisterPage	POST /api/auth/register, verify-otp, resend-otp
RF-002	Autenticación, sesión activa y recuperación de contraseña	LoginPage, ForgotPasswordPage, UserProfile, AuthContext	POST /api/auth/login, GET /api/auth/me, POST /api/auth/forgot-password, reset-password
RF-003	Asignación y modificación de roles	AdminPanel	GET /api/users, PATCH /api/users/:id/role, PATCH /api/users/:id/status
RF-004	Crear equipo (queda como capitán)	MyTeam	POST /api/teams
RF-005	Agregar, editar y dar de baja jugadores	MyTeam	POST/PUT/DELETE /api/teams/:id/players
RF-006	Elegibilidad: estudiante activo y un solo equipo por disciplina	MyTeam	Validación en POST /api/teams y POST /api/teams/:id/players
RF-007	Crear torneos	OrganizerPanel	POST /api/tournaments, PUT /api/tournaments/:id

RF	Descripción	Componente frontend (React)	Endpoint / servicio backend
RF-008	Inscripción solo dentro del período habilitado	MyTeam, OrganizerPanel	POST /api/tournaments/:id/registrations, PATCH .../registrations/:regId
RF-009	Calendario automático (liga o eliminación)	OrganizerPanel, Calendar	POST /api/tournaments/:id/fixture, .../fixture/publish
RF-010	Reprogramar partidos (fecha, hora, escenario)	OrganizerPanel	PATCH /api/matches/:id/reschedule, PATCH /api/matches/:id/referee
RF-011	Registrar marcador y estadísticas	RefereePanel	PUT /api/matches/:id/result, GET /api/matches/mine
RF-012	Tabla de posiciones automática	Standings	GET /api/matches/standings/:torneoId
RF-013	Estadísticas individuales y goleadores	Standings, UserProfile	GET /api/matches/scorers/:torneoId, GET /api/matches/players/:userId/stats
RF-014	Notificaciones por correo (inscripciones y reprogramaciones)	(transparente para el usuario)	server/services/mailer.js, invocado desde reviewRegistration, registerTeam, rescheduleMatch
RF-015	Portal público y exportación a PDF	Home, Tournaments, Calendar, Standings, botón «Exportar PDF»	GET /api/tournaments, GET /api/matches, GET /api/reports/tournaments/:id/pdf

3.2 Casos de uso

CU	Caso de uso	RF	Pantalla
CU-01	Registrarse	RF-001	Registro
CU-02	Iniciar sesión	RF-002	Login, Recuperar contraseña
CU-03	Crear equipo	RF-004	Mi equipo
CU-04	Gestionar plantilla	RF-005, RF-006	Mi equipo
CU-05	Inscribir equipo en torneo	RF-008	Mi equipo
CU-06	Crear torneo	RF-007	Panel del organizador
CU-07	Generar calendario	RF-009	Panel del organizador
CU-08	Reprogramar partido	RF-010	Panel del organizador
CU-09	Registrar resultado	RF-011, RF-012	Panel del árbitro
CU-10	Consultar posiciones y estadísticas	RF-012, RF-013, RF-015	Posiciones, Calendario
CU-11	Gestionar usuarios y roles	RF-003	Panel de administración
CU-12	Exportar reporte	RF-015	Panel del organizador, Posiciones
CU-13	Enviar notificación	RF-014	Servicio de correo

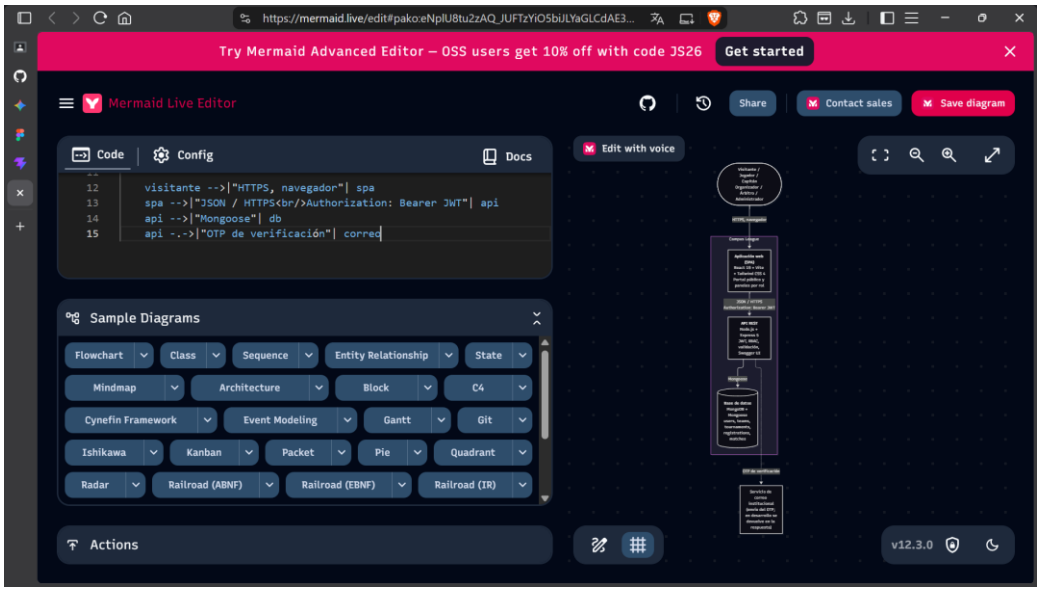
3.3 Justificación de requisitos no funcionales

RNF	Requisito	Cómo se cumple	Evidencia
RNF-001	API < 200 ms; páginas < 2 s	Índices en las consultas frecuentes (sección 5), paginación en listados, cálculo de posiciones con una consulta, build de Vite (JS ~124 kB gzip)	Los logs de desarrollo muestran 3–7 ms en listados de torneos, 9–35 ms en partidos y posiciones y ~100–180 ms en login (el costo es el hash bcrypt, deliberado)
RNF-002	JWT de 30 min, bcrypt y cifrado AES-256 de datos sensibles	JWT con JWT_EXPIRE=30m; contraseñas con bcryptjs (salt >= 10); OTP guardado como hash; el código estudiantil se cifra en la aplicación con **AES-256-GCM** (clave ENCRYPTION_KEY de 256 bits derivada con HKDF, IV derivado con HMAC-SHA256 para conservar el índice único y la búsqueda exacta, etiqueta de autenticación que detecta alteraciones). MongoDB Atlas añade cifrado del almacenamiento en reposo	server/services/crypto.js, server/models/User.js, pruebas crypto.test.js
RNF-003	Comunicación sobre HTTPS (TLS 1.3)	La app no termina TLS: se despliega detrás de un proxy o plataforma con HTTPS. helmet aplica cabeceras seguras y CORS se limita a FRONTEND_URL	Configuración en server/server.js
RNF-004	WCAG 2.1 AA	Tema oscuro con texto claro sobre fondos oscuros (contraste alto), etiquetas label, atributos aria-*, roles alert/status, foco visible, navegación por teclado y mensajes de error claros	Capturas de la sección 6
RNF-005	Responsive 375–1440 px	Diseño con utilidades responsive de Tailwind, menú hamburguesa, tablas con scroll horizontal	Navbar y paneles
RNF-006	Disponibilidad 99,5 % y respaldos diarios	Depende del despliegue: GET /api/health permite monitoreo y MongoDB Atlas ofrece respaldos automáticos	Endpoint de salud
RNF-007	500 usuarios concurrentes	API sin estado (JWT, sin sesiones en memoria), consultas indexadas, paginación y límite de intentos en autenticación	Arquitectura de la sección 4
RNF-008	Arquitectura modular y OpenAPI 3.0	Capas models, controllers, routes, middlewares, services; Swagger UI con los 44 endpoints	server/swagger/swagger.yaml

4. Arquitectura del sistema

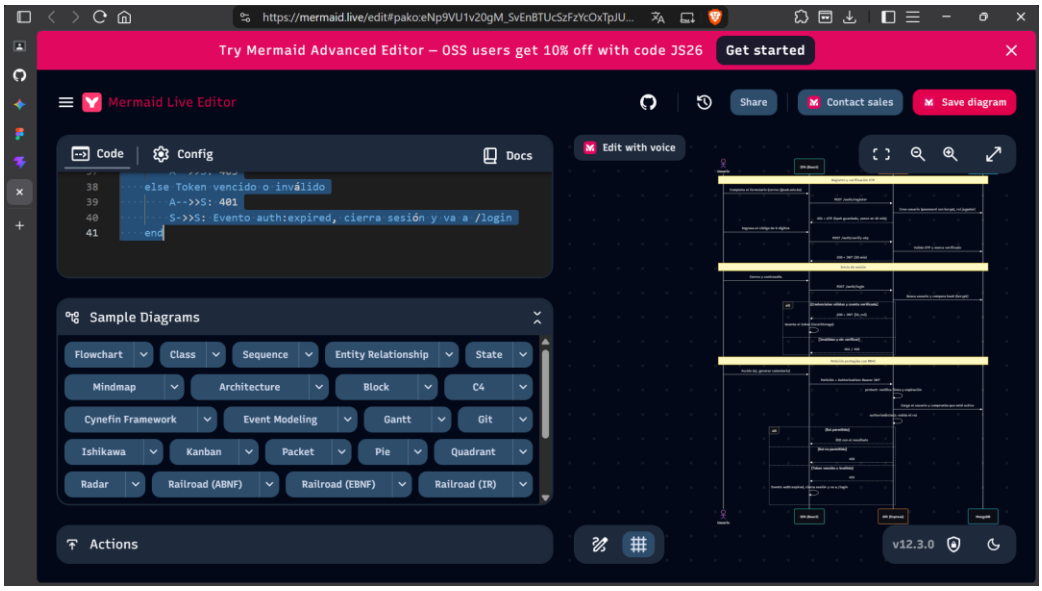
Monorepo con cliente (SPA), API REST y base de datos documental. Los diagramas fuente están en docs/diagrams/.

4.1 Diagrama C4 nivel 2 (contenedores)



La SPA (React) se comunica con la API REST (Express) por JSON/HTTPS enviando `Authorization: Bearer <JWT>`. La API persiste en MongoDB mediante Mongoose y envía correos (OTP y notificaciones) a través de un servidor SMTP configurable. Código fuente: `diagrams/c4-nivel2-contenedores.md`.

4.2 Diagrama de secuencia de autenticación y autorización



El flujo cubre el registro con OTP, el login (verificación bcrypt y firma del JWT con `id` y `rol`), el almacenamiento del token en el cliente (`localStorage`), el envío en el encabezado `Authorization: Bearer` y las validaciones de `protect` (firma, expiración, cuenta activa) y `authorize(roles)`. Un 401 hace que el cliente borre el token y vuelva a `/login`. Código fuente: `diagrams/secuencia-jwt.md`.

5. Diseño de persistencia en MongoDB

Base de datos `campus_league` con cinco colecciones y Mongoose como ODM. El diagrama entidad-relación está en `diagrams/modelo-datos.md`.

5.1 Colecciones y relaciones

Colección	Contenido	Relaciones (referencias `ObjectId`)
`users`	Cuenta, rol, facultad, código estudiantil, OTP hasheado	—

Colección	Contenido	Relaciones (referencias ObjectId)
teams	Equipo y plantilla (jugadores[] embebido)	capitan → users; jugadores[].usuario → users
tournaments	Torneo, fechas, cupos, reglamento, estado	organizador → users
registrations	Inscripción de un equipo a un torneo	torneo, equipo, inscritoPor, revisadoPor
matches	Partido, marcador, acta (estadisticas[] embebido), historial de reprogramación	torneo, local, visitante, arbitro

Se embebieron la plantilla, las estadísticas por partido y el historial porque se leen siempre junto al documento padre; se usaron referencias donde la entidad tiene vida propia (usuarios, equipos, torneos).

5.2 Validaciones de esquema

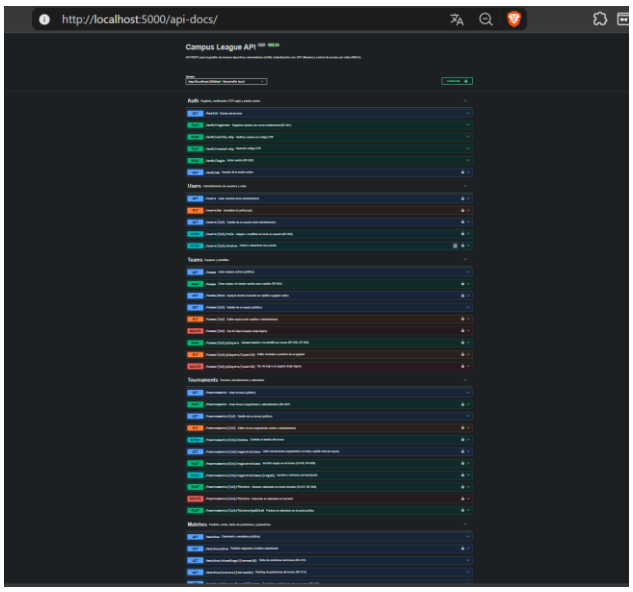
Modelo	Validaciones
User	Correo con regex @uab.edu.bo (único), contraseña mínima de 8 caracteres y select: false, rol en enum de 5 valores, código estudiantil con regex, hash bcrypt con salt >= 10 en pre('save'), código estudiantil cifrado con AES-256-GCM (setter/getter transparente), toJSON sin password ni otp
Team	Nombre 3–60, disciplina en enum, capitán obligatorio, sin jugadores repetidos, sin camisetas repetidas entre activos, máximo 30 jugadores
Tournament	Fecha de fin >= inicio, fin de inscripción >= inicio, 4–64 equipos, formato y estado en enum, reglamento obligatorio
Registration	aceptaReglamento debe ser true, estado en enum
Match	Local distinto de visitante, marcador >= 0, estado y fase en enum, goles individuales coherentes con el marcador

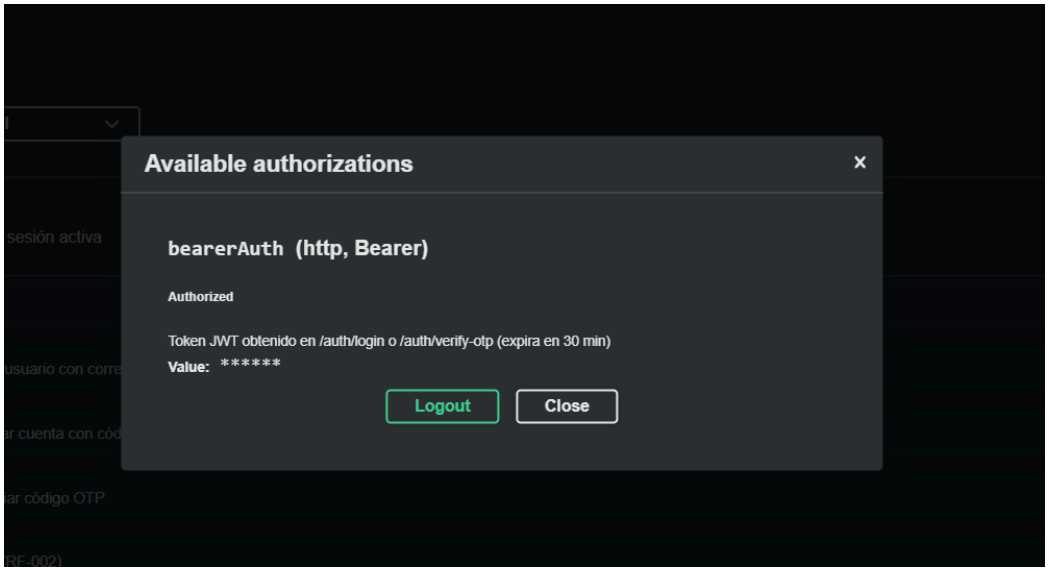
5.3 Índices

Colección	Índices
users	correo único, codigoEstudiantil único y disperso, rol, facultad + rol, createdAt
teams	nombre + disciplina único, capitan, disciplina + activo, jugadores.usuario + disciplina
tournaments	nombre + disciplina único, disciplina + estado, organizador, fechaInicio
registrations	torneo + equipo único, torneo + estado, equipo, inscritoPor
matches	torneo + fecha + escenario único, torneo + fecha, torneo + estado, local, visitante, arbitro + estado

6. Especificación de endpoints y API REST

La documentación interactiva está en **Swagger UI**: http://localhost:5000/api-docs (JSON en /api-docs.json; fuente en server/swagger/swagger.yaml). Todos los endpoints cuelgan de /api. Las respuestas de error usan el formato { "success": false, "message": "...", "errors": [...] }.





6.1 Matriz de endpoints

Códigos de uso común: 200 OK, 201 creado, 400 datos inválidos, 401 sin sesión o token inválido, 403 rol sin permiso, 404 no encontrado, 409 conflicto (duplicado o choque de horario), 429 demasiados intentos. Un «?» junto a un parámetro indica que es opcional.

Método	Ruta	Parámetros	Códigos HTTP	Acceso (RBAC)
POST	/api/auth/register	cuerpo: nombre, correo, password	201, 400, 409	Público
POST	/api/auth/verify-otp	cuerpo: correo, codigo	200, 400	Público
POST	/api/auth/resend-otp	cuerpo: correo	200, 400	Público
POST	/api/auth/login	cuerpo: correo, password	200, 400, 401, 403	Público
GET	/api/auth/me	–	200, 401	Sesión
POST	/api/auth/forgot-password	cuerpo: correo	200, 400, 429	Público
POST	/api/auth/reset-password	cuerpo: correo, codigo, password	200, 400, 429	Público
GET	/api/teams	disciplina?; facultad?; q?	200	Público
POST	/api/teams	cuerpo: nombre, disciplina	201, 400, 401, 403, 409	Jugador, Capitán, Admin
GET	/api/teams/mine	–	200, 401	Sesión
GET	/api/teams/{id}	id	200, 400, 404	Público

Método	Ruta	Parámetros	Códigos HTTP	Acceso (RBAC)
PUT	/api/teams/{id}	id	200, 401, 403, 404	Capitán del equipo, Admin
DELETE	/api/teams/{id}	id	200, 401, 403, 404	Capitán del equipo, Admin
POST	/api/teams/{id}/players	id; cuerpo: correo	201, 400, 401, 403, 404, 409	Capitán del equipo, Admin
PUT	/api/teams/{id}/players/{userId}	id; userId	200, 401, 403, 404	Capitán del equipo, Admin
DELETE	/api/teams/{id}/players/{userId}	id; userId	200, 400, 401, 403, 404	Capitán del equipo, Admin
GET	/api/tournaments	disciplina?; estado?	200	Público
POST	/api/tournaments	cuerpo: nombre, disciplina, formato, fechaInicio, fechaFin, inscripcionInicio, inscripcionFin, maxEquipos, reglamento	201, 400, 401, 403	Organizador, Admin
GET	/api/tournaments/{id}	id	200, 404	Público
PUT	/api/tournaments/{id}	id	200, 400, 401, 403, 404	Organizador dueño, Admin
PATCH	/api/tournaments/{id}/status	id; cuerpo: estado	200, 401, 403, 404	Organizador dueño, Admin
GET	/api/tournaments/{id}/registrations	id; estado?	200, 401, 403	Organizador dueño, Admin; Capitán (las suyas)
POST	/api/tournaments/{id}/registrations	id; cuerpo: equipo, aceptaReglamento	201, 400, 401, 403, 404, 409	Capitán, Admin
PATCH	/api/tournaments/{id}/registrations/{regId}	id; regId; cuerpo: estado	200, 400, 401, 403, 404, 409	Organizador dueño, Admin
POST	/api/tournaments/{id}/fixture	id	201, 400, 401, 403, 409	Organizador dueño, Admin
DELETE	/api/tournaments/{id}/fixture	id	200, 400, 401, 403	Organizador dueño, Admin
POST	/api/tournaments/{id}/fixture/publish	id	200, 400, 401, 403	Organizador dueño, Admin
GET	/api/matches	torneo?; equipo?; estado?; fase?; jornada?; desde?; hasta?; limit?	200, 400	Público (borrador solo organizador)
GET	/api/matches/mine	estado?	200, 401, 403	Árbitro, Admin
GET	/api/matches/standings/{torneoId}	torneoId; torneoId?	200, 400, 404	Público
GET	/api/matches/scorers/{torneoId}	torneoId; torneoId?; limit?	200, 400	Público
GET	/api/matches/players/{userId}/stats	userId; torneo?	200, 400	Público
GET	/api/matches/{id}	id	200, 404	Público (borrador solo organizador)
PATCH	/api/matches/{id}/referee	id; cuerpo: arbitro	200, 400, 401, 403, 404, 409	Organizador dueño, Admin
PATCH	/api/matches/{id}/reschedule	id	200, 400, 401, 403, 404, 409	Organizador dueño, Admin
PATCH	/api/matches/{id}/reopen	id	200, 400, 401, 403, 404	Organizador dueño, Admin
PUT	/api/matches/{id}/result	id	200, 400, 401, 403, 404, 409	Árbitro asignado, Admin
GET	/api/users	rol?; activo?; q?; page?; limit?	200, 401, 403	Admin
PUT	/api/users/me	–	200, 400, 401	Sesión
GET	/api/users/{id}	id	200, 401, 403, 404	Admin
PATCH	/api/users/{id}/role	id; cuerpo: rol	200, 400, 401, 403, 404	Admin

Método	Ruta	Parámetros	Códigos HTTP	Acceso (RBAC)
PATCH	/api/users/{id}/status	id; cuerpo: activo	200, 400, 401, 403, 404	Admin
GET	/api/users/referees	–	200, 401, 403	Organizador, Admin
GET	/api/reports/tournaments/{id}/pdf	id; tipo?	200, 400, 401, 403, 404	Organizador dueño, Admin

6.2 Seguridad aplicada a la API

- protect verifica firma y expiración del JWT y que la cuenta siga activa; authorize(...roles) aplica el RBAC.
- El rol nunca se acepta desde el cliente: todo registro inicia como jugador.
- Datos personales sensibles (código estudiantil) cifrados con AES-256-GCM antes de guardarse; la API los devuelve descifrados solo a quien tiene acceso.
- Validación de entrada con express-validator en todas las rutas, helmet, CORS restringido y límite de intentos en /api/auth.
- Respuestas genéricas en resend-otp y forgot-password para no revelar si un correo existe.

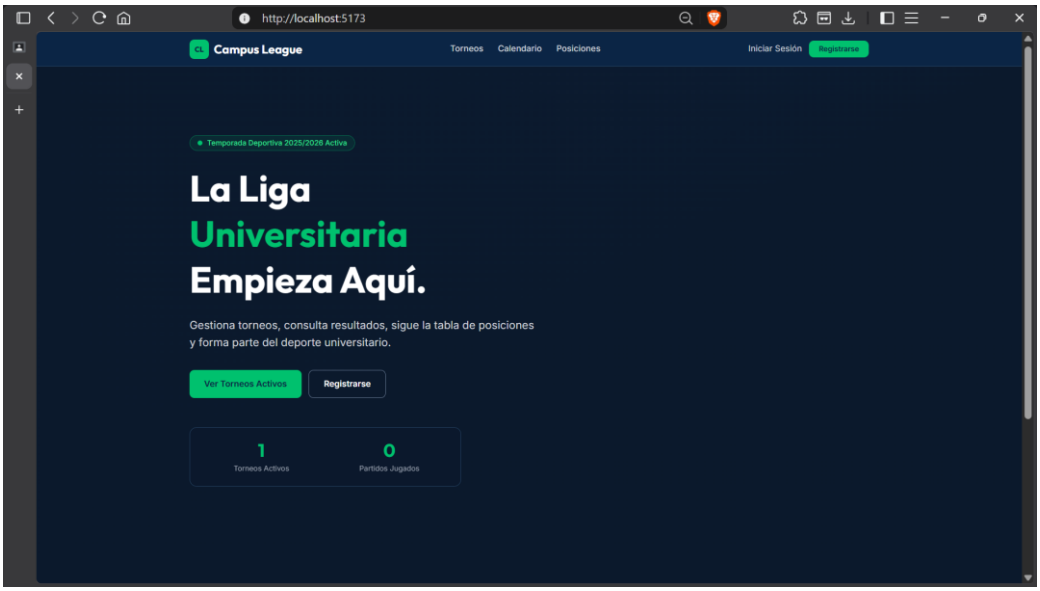
6.3 Reglas de negocio

- Plantilla mínima y jugadores habilitados para inscribirse; lista de espera al llenarse los cupos.
- Calendario con mínimo de 4 equipos aprobados, liga (algoritmo del círculo, ida y vuelta opcional) o eliminación directa, y detección de choques de escenario, equipos y árbitro con una ventana de 90 minutos.
- Acta cerrada solo si los goles individuales coinciden con el marcador; walkover 3-0; reapertura dentro de 24 horas.
- Reprogramar no admite fechas pasadas y conserva historial.
- Posiciones y goleadores se calculan al consultar (3 puntos por victoria, 1 por empate).

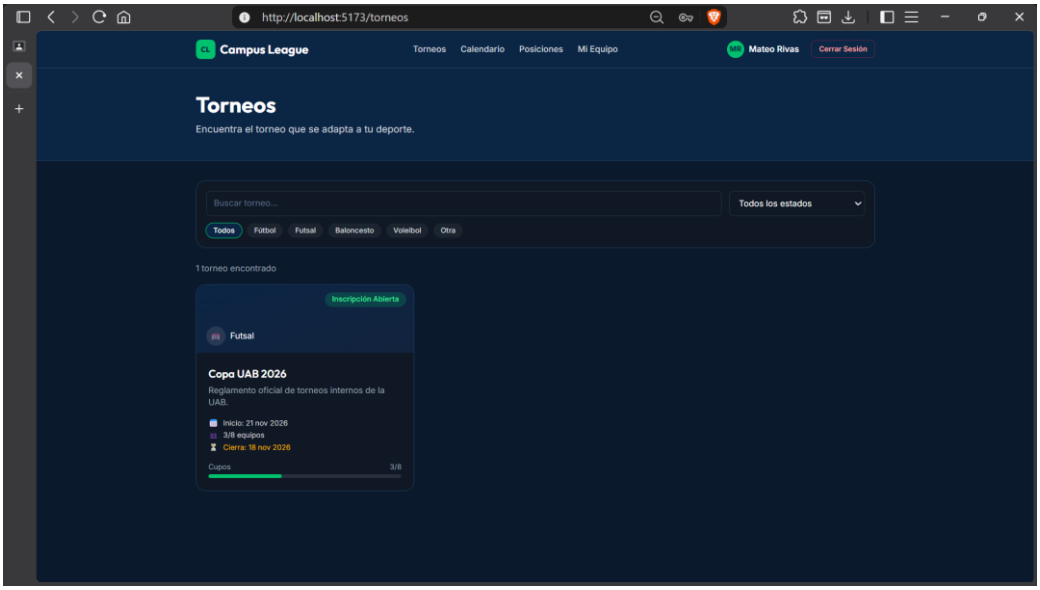
6.4 Capturas de la aplicación

Portal público

Inicio



Catálogo de torneos



Calendario de partidos

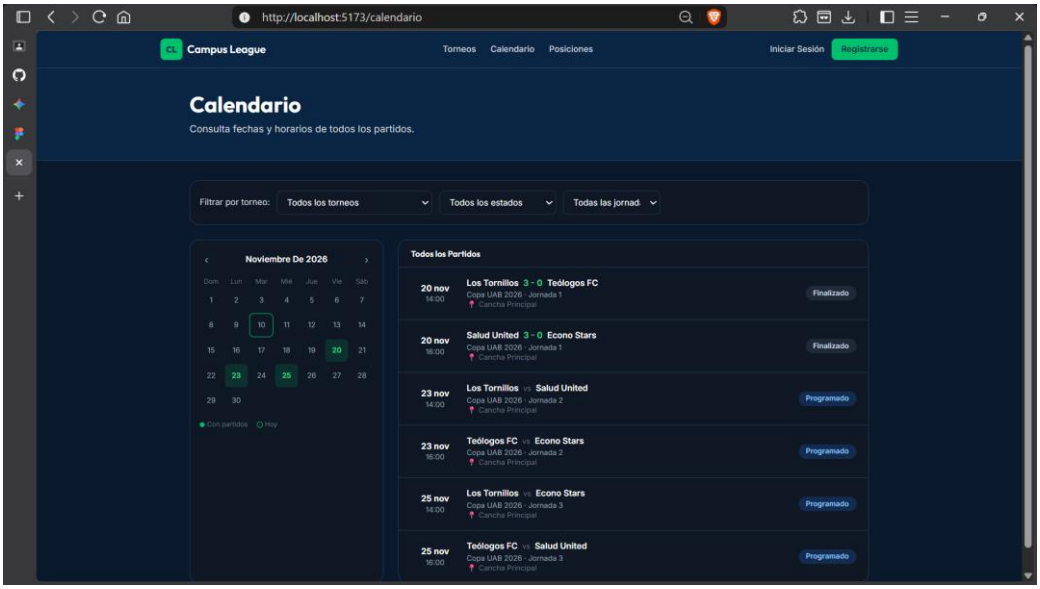
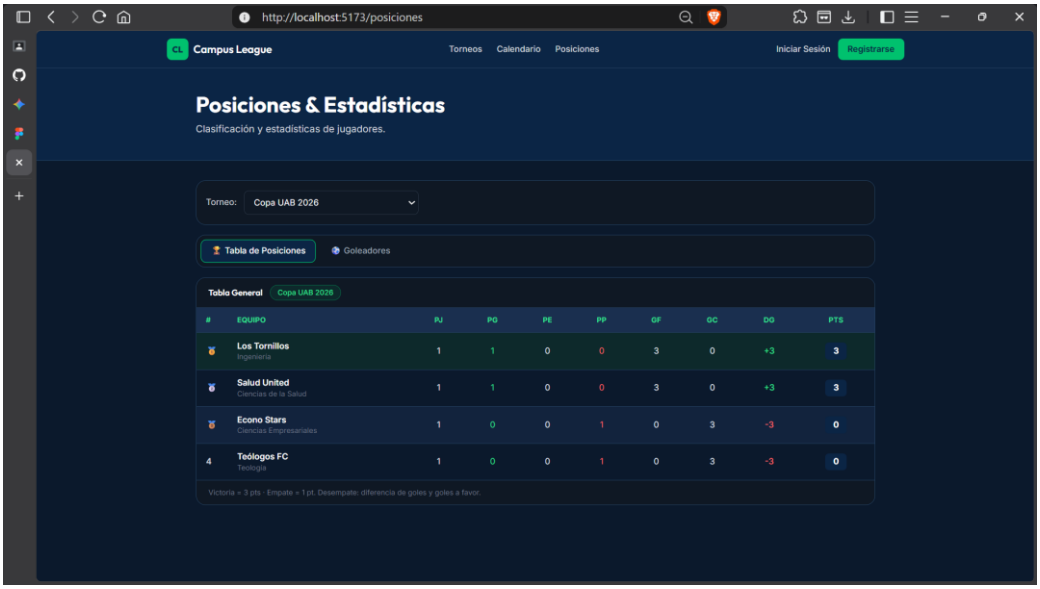
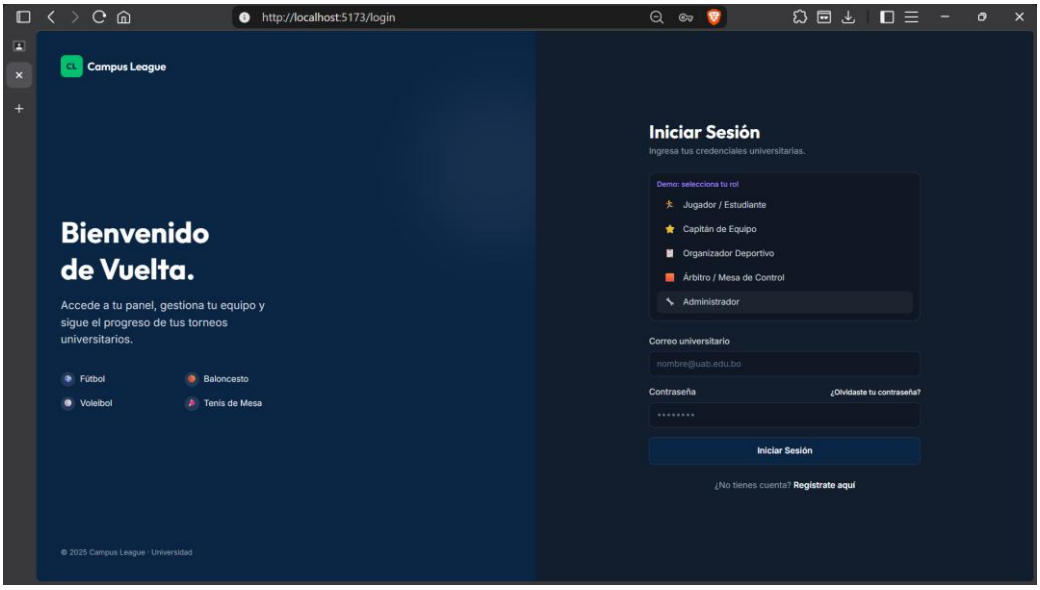


Tabla de posiciones

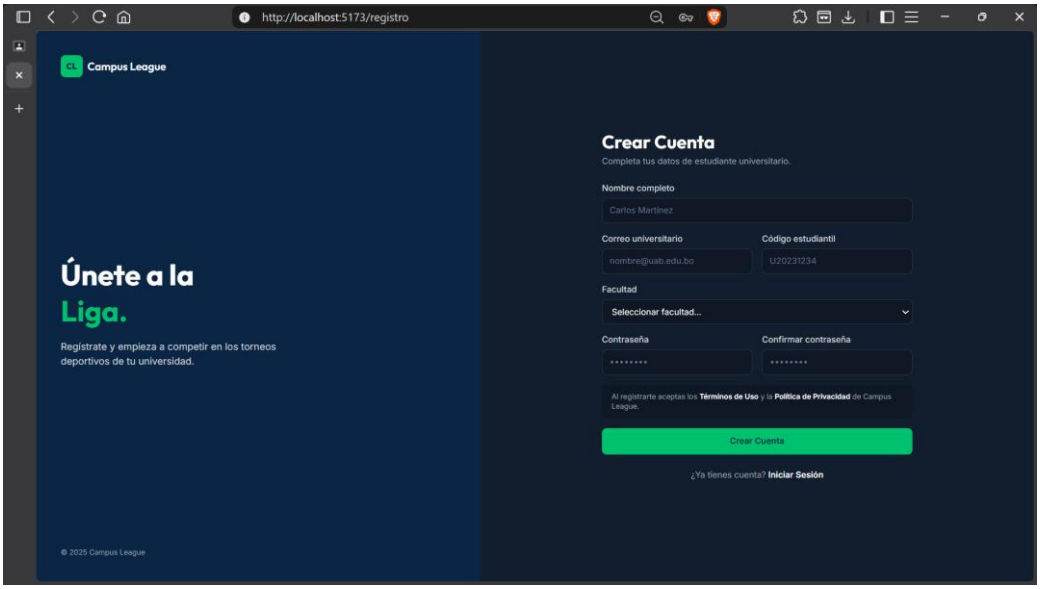


Autenticación y cuenta (RF-001, RF-002)

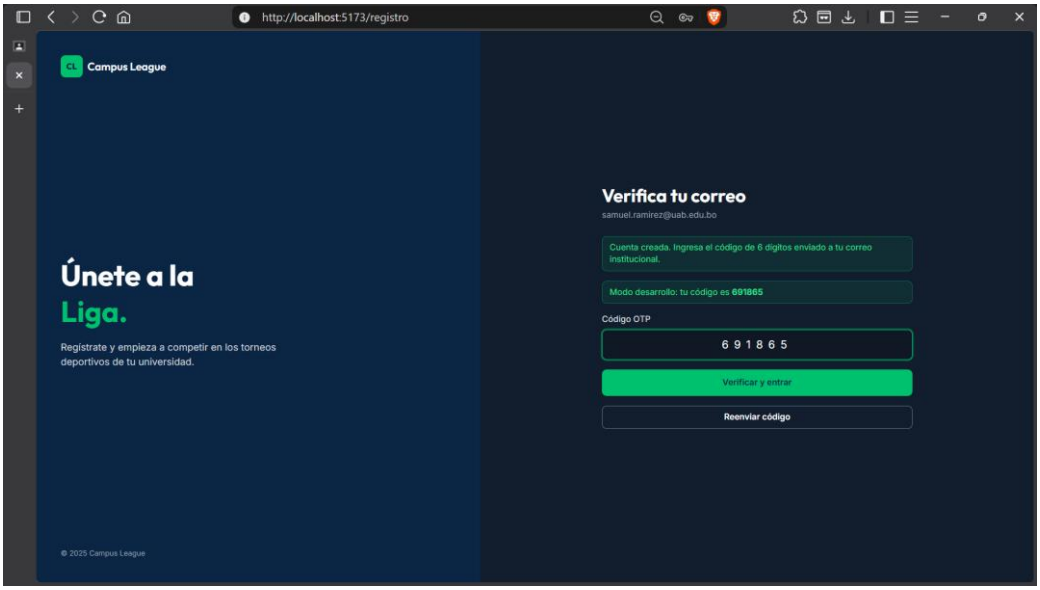
Inicio de sesión



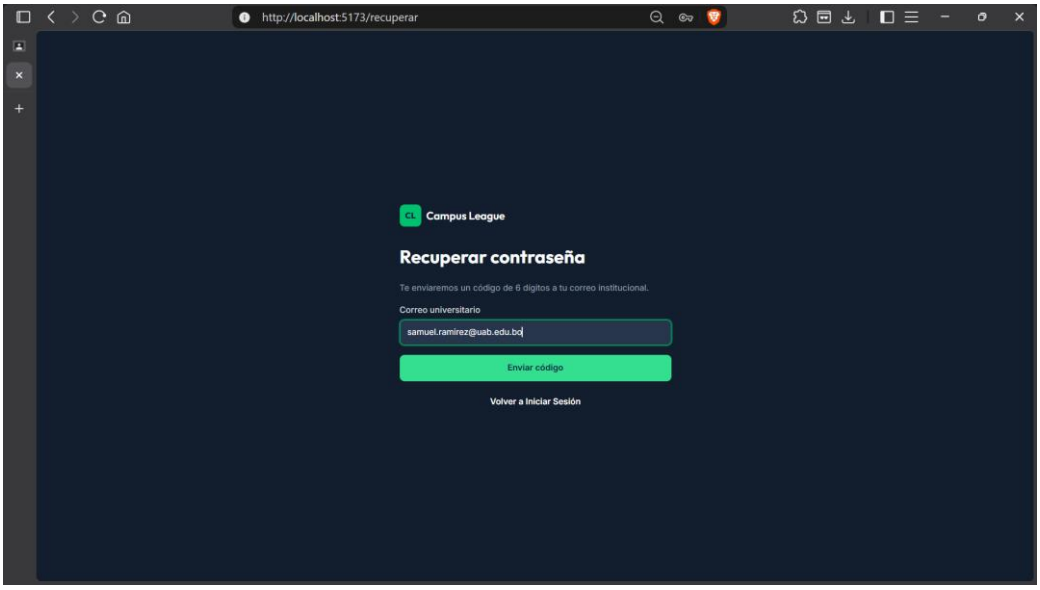
Registro con correo institucional



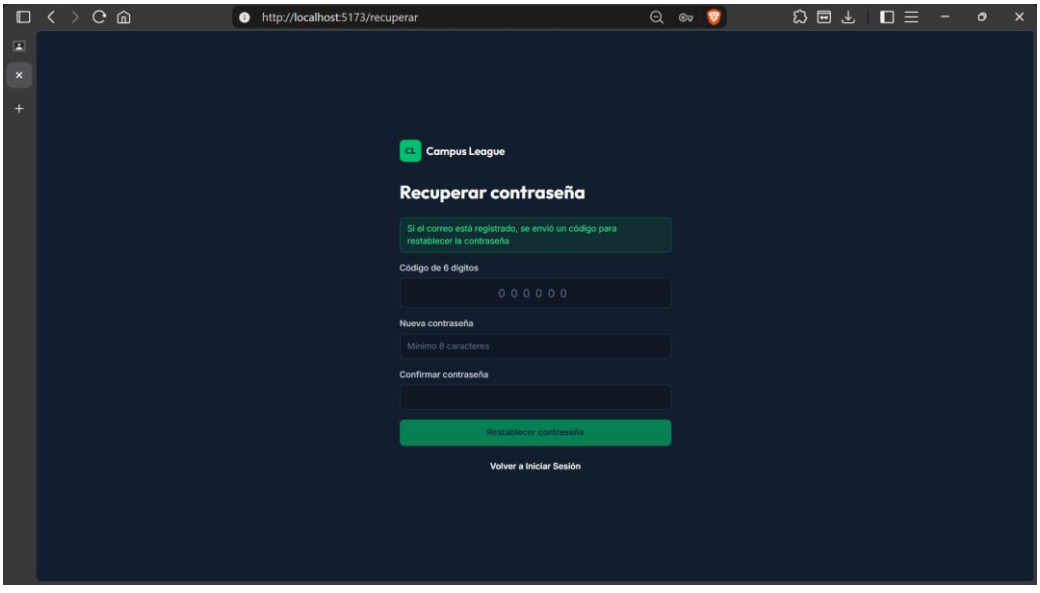
Verificación con código OTP



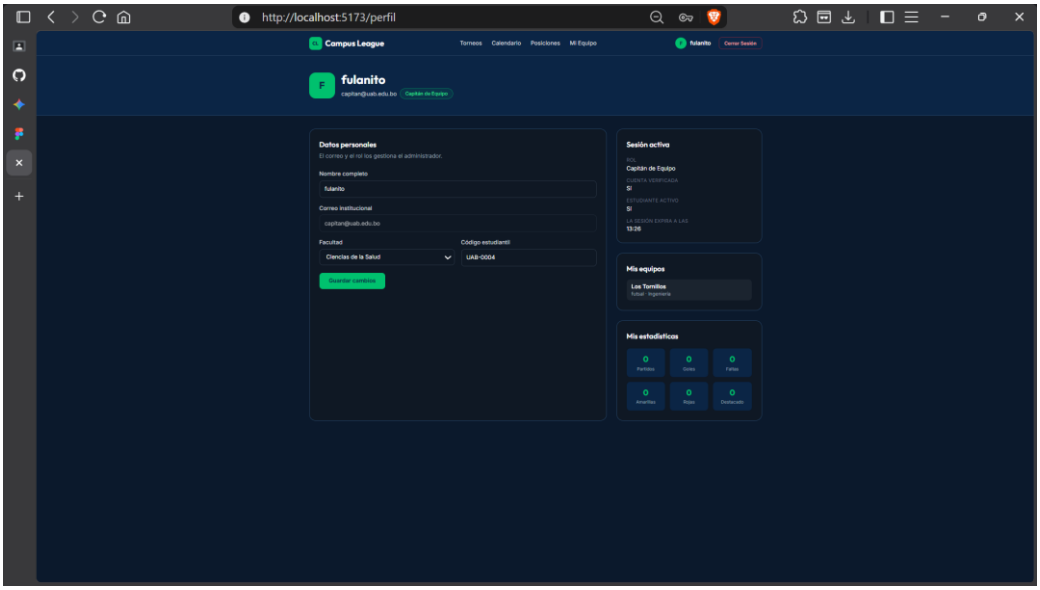
Recuperación de contraseña: solicitud del código



Recuperación de contraseña: nuevo código y contraseña

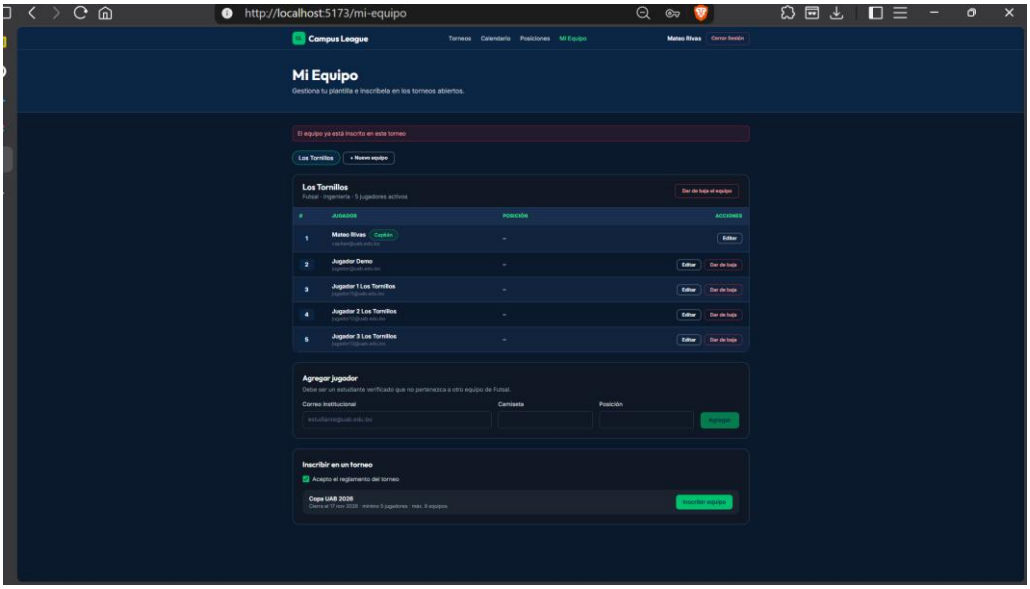


Perfil de usuario

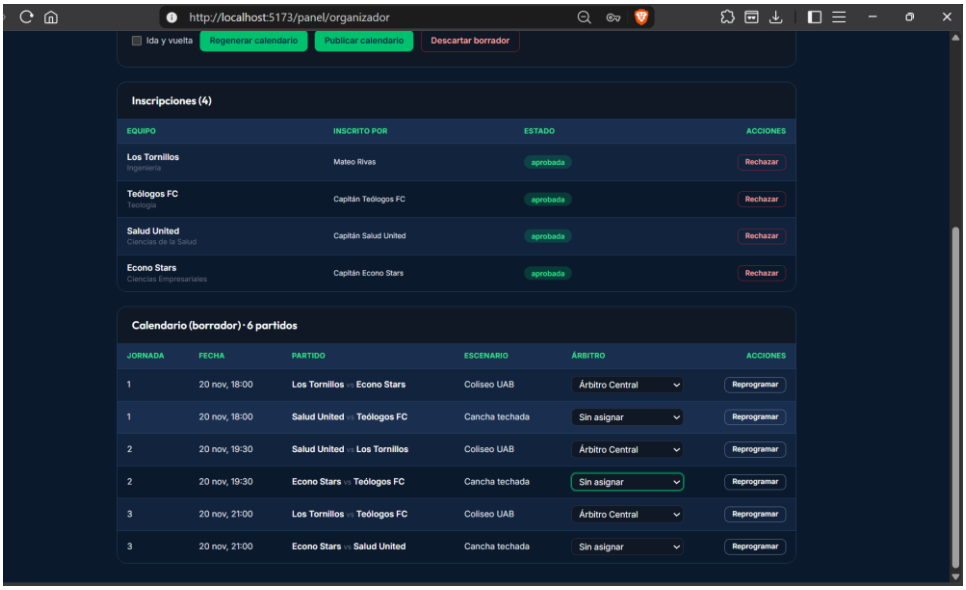


Gestión por rol

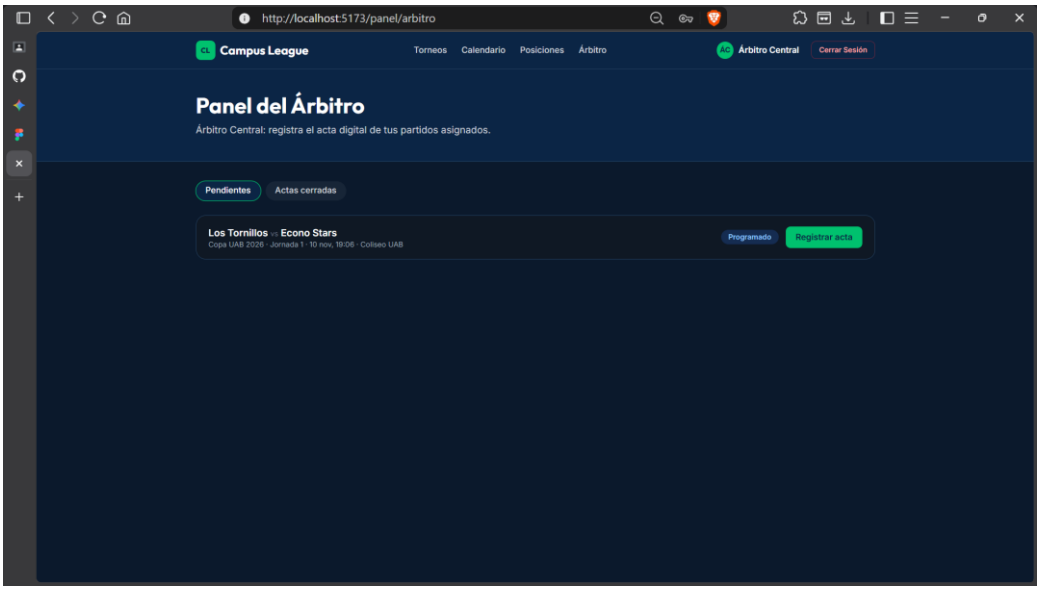
Gestión de equipo (capitán)



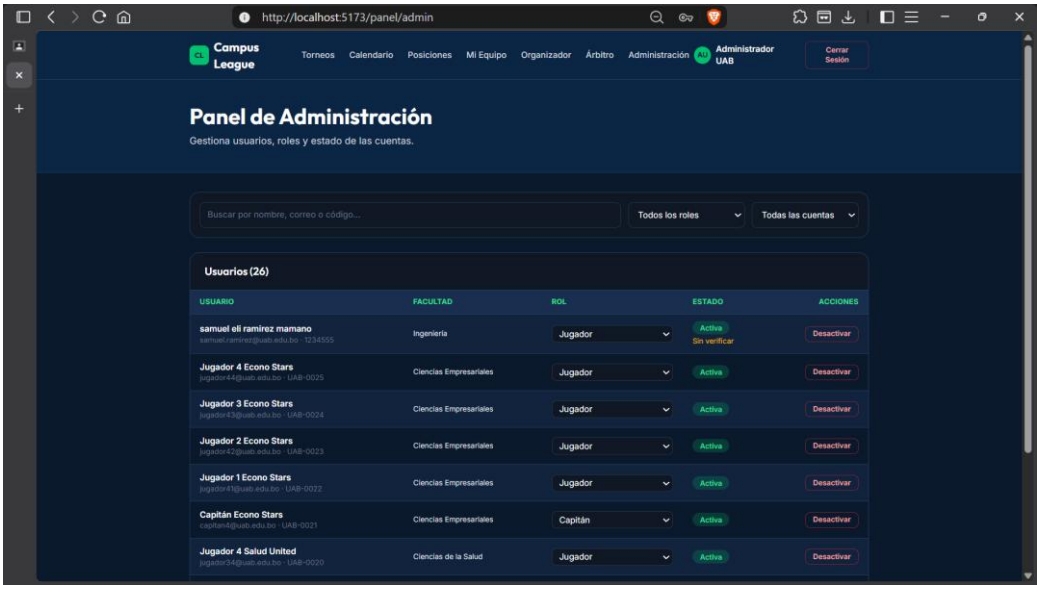
Panel del organizador



Panel del árbitro

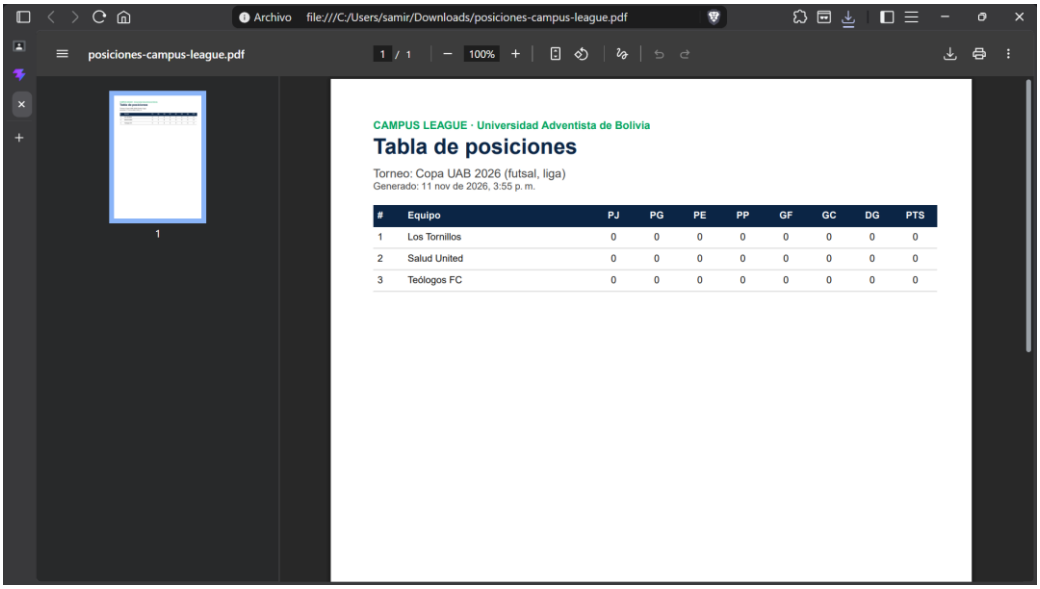


Panel de administración



Reportes y seguridad (RF-015, RNF-002)

Reporte de posiciones exportado en PDF

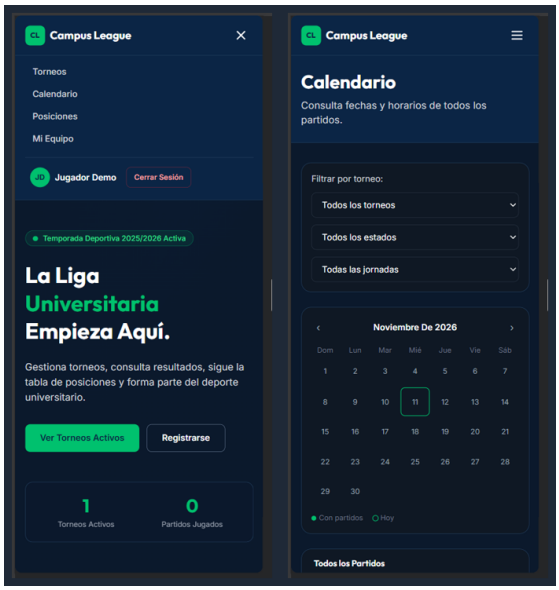


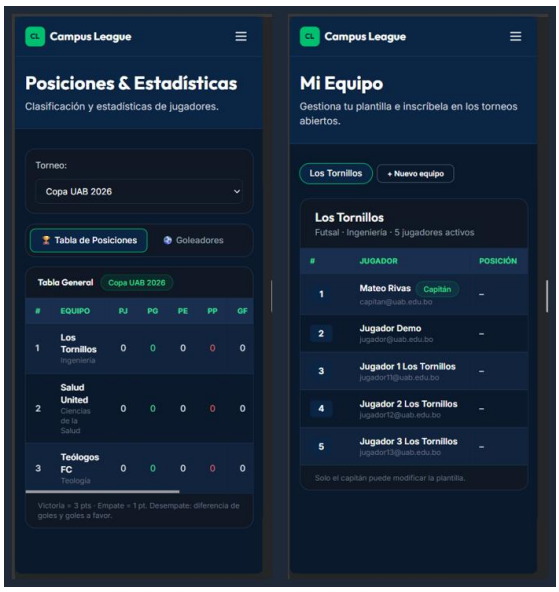
Código estudiantil cifrado con AES-256-GCM en MongoDB



6.5 Diseño responsivo (RNF-005)

La interfaz se adapta de 375 a 1440 px: en pantallas menores de 1024 px la navegación se pliega en un menú desplegable, los filtros y formularios se apilan en una columna, las tarjetas ocupan todo el ancho y las tablas se desplazan horizontalmente dentro de su contenedor sin mover la página. Debajo se muestran capturas a 375 px: menú desplegado e inicio, calendario, tabla de posiciones y gestión de equipo.





7. Configuración de entorno y variables

Los archivos .env.example (raíz, client/ y server/) documentan las variables sin credenciales reales. Cada integrante copia server/.env.example a server/.env y client/.env.example a client/.env. .env está excluido por .gitignore.

7.1 Servidor (`server/.env`)

Variable	Descripción	Local	Producción
PORT	Puerto de la API	5000	Lo asigna la plataforma
NODE_ENV	Entorno. En production el OTP no se devuelve en las respuestas	development	production
MONGO_URI	Cadena de conexión	mongodb://127.0.0.1:27017/campus_league	URI de MongoDB Atlas (mongodb+srv://...)
JWT_SECRET	Clave de firma de tokens	Cadena larga y aleatoria	Secreto único generado con crypto.randomBytes(64)
JWT_EXPIRE	Vigencia del token	30m	30m
BCRYPT_SALT_ROUNDS	Costo de bcrypt (mínimo 10)	10	10 o más
ENCRYPTION_KEY	Clave AES-256 (64 caracteres hexadecimales) para cifrar datos personales	Vacía (se deriva de JWT_SECRET)	Obligatoria: openssl rand -hex 32
FRONTEND_URL	Orígenes CORS permitidos (separados por coma)	http://localhost:5173	URL pública del cliente
API_PROD_URL	URL pública de la API para Swagger	—	https://api.midominio.com
SMTP_HOST, SMTP_PORT, SMTP_SECURE, SMTP_USER, SMTP_PASS	Servidor de correo. Sin SMTP_HOST los correos se simulan en consola	vacío	Servidor SMTP real
MAIL_FROM	Remitente de los correos	Campus League <no-reply@uab.edu.bo>	Correo institucional
RATE_LIMIT_MAX	Intentos por IP cada 15 min en /api/auth	100	Valor más estricto (p. ej. 20)

7.2 Cliente (`client/.env`)

Variable	Descripción	Local	Producción
VITE_API_URL	URL base de la API	http://localhost:5000/api	https://api.midominio.com/api

8. Manual de ejecución y pruebas

8.1 Requisitos previos

Node.js 20 o superior, npm, Git y MongoDB local (`mongod`) o una cuenta de MongoDB Atlas.

8.2 Instalación y ejecución local

```
git clone https://github.com/deaht911/campus-league.git
cd campus-league
npm run install:all
cp server/.env.example server/.env
cp client/.env.example client/.env
# editar server/.env: MONGO_URI y JWT_SECRET
npm run seed     # carga datos de demostración (borra la base)
npm run dev      # API en :5000 y cliente en :5173
```

URL	Contenido
http://localhost:5173	Aplicación
http://localhost:5000/api-docs	Swagger UI
http://localhost:5000/api/health	Estado del servicio

Cuentas de demostración (contraseña `Campus2026!`): `admin@`, `organizador@`, `arbitro@`, `capitan@` y `jugador@uab.edu.bo`.

8.3 Pruebas automáticas

```
npm test --prefix server     # 39 pruebas, no requieren MongoDB
cd client && npm run lint && npm run build
```

Archivo	Qué verifica
models.test.js	Validadores e índices de los 5 modelos, bcrypt, OTP hasheado y su vencimiento
middlewares.test.js	protect (sin token, inválido, expirado, cuenta inactiva), authorize (RBAC) y manejo de errores
api.test.js	Endpoints con Supertest: salud, Swagger, registro, login (401, 403, 200 y expiración de 30 min), /me, recuperación de contraseña, RBAC (401/403), IDs inválidos y exportación PDF
crypto.test.js	Cifrado AES-256-GCM: ida y vuelta, determinismo, detección de alteraciones, datos antiguos y modelo User
services.test.js	Envío de correos simulados y generación de PDF

Resultado de la ejecución de `npm test --prefix server`:



8.4 Pruebas manuales realizadas (sobre el seed)

Área	Resultado
Autenticación	Login rechazado sin verificar (403), verificación OTP (200), `/me` con y sin token
Usuarios	Filtros, cambio de rol, desactivación y reactivación; no se puede cambiar el propio rol
Equipos	Alta con promoción a capitán, jugador duplicado (409), baja lógica
Inscripción	Aprobar y rechazar; equipo ya inscrito (409)
Calendario	Generación de 6 partidos, publicación, borrador oculto al público
Choques de horario	Árbitro solapado (409), reprogramación a fecha pasada (400)
Acta	Goles que no cuadran (400), acta cerrada (200), cerrada no editable (409), walkover 3-0
Posiciones	Tabla y goleadores actualizados al cerrar actas
Control de acceso	Pantallas por rol («Acceso restringido») y 403 en el servidor

8.5 Flujo de trabajo Git

Ramas `feature/*` creadas desde `develop`, integración con `git merge --ff-only`, y 4 autores con commits propios. El historial completo se ve en GitHub (Insights → Contributors).

9. Conclusiones y trabajo futuro

9.1 Conclusiones técnicas

- La arquitectura desacoplada (SPA + API REST + MongoDB) permitió desarrollar y probar cada capa por separado y repartir el trabajo entre cuatro integrantes sin conflictos.
- Concentrar las reglas de negocio en el servidor (elegibilidad, cupos, choques de horario, coherencia del acta) evita inconsistencias aunque se use otro cliente. El cliente solo anticipa errores para mejorar la experiencia.
- JWT sin estado más RBAC en middleware dio una seguridad uniforme: una sola cadena `protect` → `authorize` protege todas las rutas.
- Documentar con OpenAPI desde el inicio y verificar con pruebas automáticas que Swagger publica los módulos principales ayudó a que la documentación no se desfasara del código.
- Calcular posiciones y goleadores al consultar, en lugar de almacenarlos, elimina la sincronización de datos derivados.

9.2 Lecciones aprendidas

- Las validaciones de solapamiento deben trabajar con intervalos, no con la hora exacta de inicio: la primera versión permitía asignar un árbitro a dos partidos casi simultáneos.
- Express 5 ejecuta el callback de `listen` también ante errores; conviene manejar ese caso para no mostrar un mensaje de éxito falso.
- Los datos de demostración reproducibles (`npm run seed`) aceleraron las pruebas y las capturas.
- Probar con roles distintos reveló huecos de diseño (por ejemplo, el organizador no podía listar árbitros), corregidos con un endpoint específico.

9.3 Trabajo futuro

- Ajustar la duración de los partidos por disciplina (hoy es fija, 90 minutos).
- Fases eliminatorias completas: generar rondas siguientes según los ganadores.
- Extender el cifrado AES-256 a más campos con rotación de claves y despliegue con TLS 1.3 de extremo a extremo.
- Pruebas de integración con base de datos en memoria y pruebas end-to-end (Playwright) en el cliente.
- Notificaciones adicionales (recordatorios de partido) y cola de correos con reintentos.
- Despliegue con CI/CD, monitoreo de disponibilidad y respaldos programados.